

AI Research & Document Intelligence

Complete Project Guide

Files, workflow, code reference, setup, synopsis and GitHub record

Project folder: AI Research Assistant

This document is a single reference for the project we built: PDF processing, semantic embeddings, FAISS retrieval, Gemini-based RAG, Streamlit UI, testing and GitHub upload.

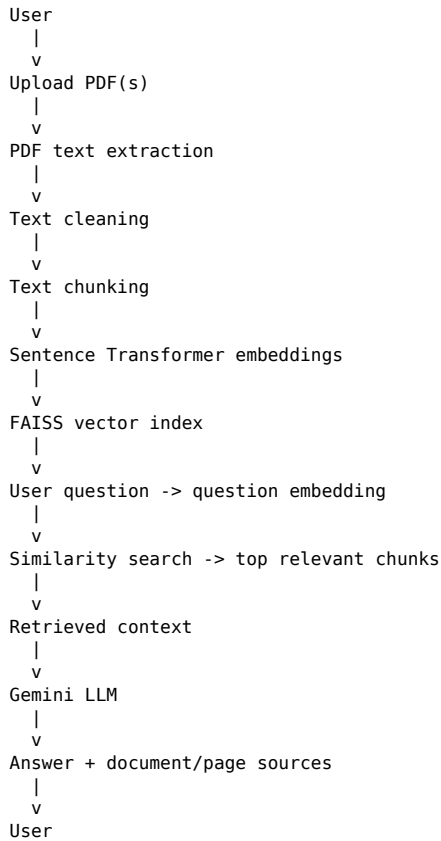
1. Final Project Structure

```
AI Research Assistant/  
├── app.py  
├── rag_pipeline.py  
├── requirements.txt  
├── .gitignore  
├── data/  
│   └── papers/  
│       └── research.pdf  
├── src/  
│   ├── pdf_processor.py  
│   ├── text_cleaner.py  
│   ├── chunker.py  
│   ├── embedder.py  
│   ├── vector_store.py  
│   ├── retriever.py  
│   └── llm.py  
├── test_pdf.py  
└── test_gemini.py
```

2. File-by-File Explanation

File	Role
app.py	Main Streamlit interface. Handles PDF upload, processing, questions, recommended questions, chat history, answer
rag_pipeline.py	Command-line RAG pipeline used for backend testing.
pdf_processor.py	Extracts PDF text page by page using PyMuPDF.
text_cleaner.py	Cleans extracted text before chunking.
chunker.py	Splits cleaned document text into smaller chunks for retrieval.
embedder.py	Creates semantic vectors using Sentence Transformers.
vector_store.py	Creates the FAISS vector index.
retriever.py	Searches FAISS and returns the most relevant chunks.
llm.py	Uses Gemini to generate an answer from the retrieved context.
test_pdf.py	Tests the PDF-to-retrieval pipeline.
test_gemini.py	Tests Gemini answer generation.
requirements.txt	Lists Python packages required by the project.
.gitignore	Keeps venv, API secrets and uploaded PDFs out of Git.

3. Complete Application Flow



4. Features Completed

- Multiple PDF upload
- Page-wise PDF text extraction
- Text cleaning and chunking
- Sentence Transformer embeddings using all-MiniLM-L6-v2
- FAISS semantic retrieval
- RAG answer generation with Gemini
- Document and page source display
- 20 recommended questions
- Chat history
- Reset Documents
- Research report download as TXT
- Streamlit UI with custom styling
- Git and GitHub setup

5. requirements.txt

```
streamlit
pymupdf
sentence-transformers
faiss-cpu
openai
python-dotenv
numpy
```

6. .gitignore

```
venv/
__pycache__/
```

```
*.pyc  
.env  
data/papers/*.pdf
```

The API key file and uploaded PDFs are intentionally excluded from the public repository.

7. app.py — Main Code

Main imports:

```
import streamlit as st
from src.pdf_processor import extract_text_from_pdf
from src.text_cleaner import clean_text
from src.chunker import create_chunks
from src.embedder import create_embeddings
from src.vector_store import create_vector_store
from src.retriever import search_similar_chunks
from src.llm import ask_llm
```

PDF processing:

```
uploaded_files = st.file_uploader(
    "Upload one or more PDF files",
    type=["pdf"],
    accept_multiple_files=True
)

for file in uploaded_files:
    temp_path = f"data/papers/{file.name}"
    with open(temp_path, "wb") as f:
        f.write(file.getbuffer())

    pages = extract_text_from_pdf(temp_path)

    for page in pages:
        all_pages.append({
            "page": page["page"],
            "text": clean_text(page["text"]),
            "document": file.name
        })

chunks = create_chunks(all_pages)
texts = [chunk["text"] for chunk in chunks]
embeddings = create_embeddings(texts)
index = create_vector_store(embeddings)
```

Question retrieval and RAG:

```
question_embedding = create_embeddings([question])

results = search_similar_chunks(
    index,
    question_embedding,
    chunks,
    top_k=5
)

context = "\n\n".join(
    [
        f"Document: {result.get('document', 'Research Paper')}\n\n",
        f"Page: {result['page']}\n\n",
        f"{result['text']}"
        for result in results
    ]
)

answer = ask_llm(question, context)
```

Chat history and report:

```
st.session_state["chat_history"].append({
    "question": question,
    "answer": answer
})

st.download_button(
    "Download Research Report",
    report,
    file_name="research_report.txt",
    mime="text/plain",
    use_container_width=True
)
```

8. src/llm.py — Gemini Code

```
import os
from dotenv import load_dotenv
from google import genai

load_dotenv()

api_key = os.getenv("GEMINI_API_KEY")
if not api_key:
    raise ValueError("GEMINI_API_KEY not found in .env file")

client = genai.Client(api_key=api_key)

def ask_llm(question, context):
    prompt = f"""
    You are an AI research assistant.

    Answer the user's question using ONLY the information
    provided in the research document context below.

    If the answer is not present in the context, say:
    "I could not find this information in the provided document."

    Research Context:
    {context}

    User Question:
    {question}

    Give a clear and concise answer.
    """

    response = client.models.generate_content(
        model="gemini-3.6-flash",
        contents=prompt
    )
    return response.text
```

Keep GEMINI_API_KEY only in .env. Never publish the key on GitHub.

9. test_pdf.py

```
from src.pdf_processor import extract_text_from_pdf
from src.text_cleaner import clean_text
from src.chunker import create_chunks
from src.embedder import create_embeddings
from src.vector_store import create_vector_store
from src.retriever import search_similar_chunks

pdf_path = "data/papers/research.pdf"

pages = extract_text_from_pdf(pdf_path)

for page in pages:
    page["text"] = clean_text(page["text"])

chunks = create_chunks(pages)
texts = [chunk["text"] for chunk in chunks]
embeddings = create_embeddings(texts)
index = create_vector_store(embeddings)

question = "What are the main objectives of this project?"
question_embedding = create_embeddings([question])

results = search_similar_chunks(
    index,
    question_embedding,
    chunks,
    top_k=3
)

for i, result in enumerate(results):
    print("RESULT:", i + 1)
    print("PAGE:", result["page"])
    print("DISTANCE:", result["distance"])
    print("TEXT:")
    print(result["text"])
```

10. test_gemini.py

```
from src.llm import ask_llm

question = "What is the main purpose of this project?"
```

```
context = """
This project is an AI-based research assistant that reads
PDFs, research papers, reports, and notes. It uses ML,
LLMs, and RAG to find useful information and answer
questions from documents.
"""

answer = ask_llm(question, context)

print("\nQUESTION:")
print(question)

print("\nANSWER:")
print(answer)
```

11. Backend Modules — Simple Explanation

pdf_processor.py: PDF → page-wise text.

text_cleaner.py: Raw text → cleaner text.

chunker.py: Long text → smaller chunks.

embedder.py: Text → numerical semantic vectors.

vector_store.py: Vectors → FAISS index.

retriever.py: Question vector → relevant chunks.

llm.py: Question + retrieved context → Gemini answer.

12. Development Result

The backend was successfully tested through extraction, chunking, embeddings, FAISS retrieval and Gemini answer generation. The tested run produced 6 chunks, created embeddings and a FAISS index, and displayed source pages.

13. Run the Project

```
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
streamlit run app.py
```

The Streamlit app runs locally using the localhost address shown by the terminal.

14. GitHub Upload — Commands Completed

```
git init
git add .
git commit -m "Initial project upload"
git branch -M main
git remote add origin "YOUR-GITHUB-REPOSITORY-URL"
git push -u origin main --force
```

The final push succeeded and the main branch is tracking origin/main.

Repository: <https://github.com/avinashshere17-cmyk/AI-Research---Document-Intelligence>

15. Project Synopsis

Introduction: AI Research & Document Intelligence is an AI-powered research assistant for interacting with research papers and PDF documents using natural-language questions. It combines document processing, semantic search, vector databases, RAG and an LLM.

Problem Statement: Large research documents are time-consuming to search manually. Keyword search may miss meaning. The system retrieves semantically relevant sections and generates concise grounded answers.

Aim: Develop an AI-based document intelligence system that accepts research PDFs, retrieves relevant content using semantic similarity and generates answers with source/page references.

Objectives: Process multiple PDFs; extract and clean text; create chunks; generate embeddings; store/search embeddings using FAISS; retrieve relevant chunks; use Gemini with RAG; display sources; maintain chat history; provide recommended questions and report export.

Use Case: Students, researchers, teachers and professionals can upload research papers and ask questions about their contents.

Tech Stack: Python, Streamlit, PyMuPDF, Sentence Transformers (all-MiniLM-L6-v2), FAISS, Gemini, RAG and VS Code.

Expected Output: Interactive PDF upload, question answering, recommended questions, chat history, source/page display and downloadable research report.

Advantages: Faster document search, semantic retrieval, multi-PDF support, grounded answers and traceable sources.

Limitations: Answer quality depends on document quality, retrieved chunks and the language model. Complex scanned PDFs, images and tables may need additional processing.

Future Scope: OCR, table/image understanding, stronger citation verification, more document formats, paper comparison, authentication, cloud deployment and multilingual support.

16. Final Project Checklist

- ✓ Project folder created
- ✓ Python virtual environment created
- ✓ Dependencies installed
- ✓ PDF extraction tested
- ✓ Embeddings tested
- ✓ FAISS retrieval tested
- ✓ Gemini integration tested
- ✓ Streamlit application completed
- ✓ Multiple PDF support added
- ✓ Recommended questions added
- ✓ Chat history added
- ✓ Source display added
- ✓ Report download added

- ✓ Git initialized
- ✓ Project committed
- ✓ GitHub remote connected
- ✓ Project pushed to GitHub

17. Important Security Note

The `.env` file contains the Gemini API credential and is excluded through `.gitignore`. Do not upload it to a public GitHub repository. Uploaded research PDFs are also excluded by the current `.gitignore` rule.

This guide consolidates the project material captured during development. Where an exact full source file was not available in the conversation file store, the document gives the verified file purpose rather than inventing unseen code.